Used Car Price Prediction

Final Year MCA Project — Guide Submission Document

Student	Mangalaparthi Sai Krishna
Program	Master of Computer Applications (MCA) — Final Year
College	Osmania University, Hyderabad
Project	Used Car Price Prediction using Machine Learning
Tech Stack	Python · Django · Scikit-learn · HTML/CSS/JS · SQLite
LinkedIn	linkedin.com/in/mangalaparthi-sai-krishna

1. Project Overview

What is this project?

This project is a web-based application that predicts the resale price of used cars in India using Machine Learning. A user enters car details — model name, year of manufacture, kilometres driven, fuel type, seller type, transmission, and owner type — and the system returns a price estimate in Indian Rupees (Lakhs) instantly.

Why this problem?

The used car market in India is worth over ■1.5 lakh crore and growing. Buyers and sellers frequently overpay or underprice vehicles because there is no transparent, data-driven pricing standard. This project addresses that gap by using historical data and regression models to produce objective price estimates.

Dataset

Source: Kaggle — 'Vehicle dataset from CarDekho' | Rows: ~8,128 entries | Features: Car Name, Year, Selling Price, Km Driven, Fuel Type, Seller Type, Transmission, Owner | Target Variable: Selling Price (in Lakhs)

Project Objectives

1. Train a regression model to predict used car prices with high accuracy.
2. Build a user-friendly Django web application to take inputs and display predictions.
3. Handle edge cases such as unknown car models gracefully.
4. Deploy a visually polished, mobile-responsive interface.

5. Demonstrate real-world applicability of ML in the Indian automotive market.

2. Technology Stack — What, Why, and How

Technology	Role in Project	Why This Choice
Python 3.x	Core language	Industry standard for ML; rich library ecosystem
Pandas	Data loading, cleaning, EDA	Best tool for tabular data manipulation
NumPy	Numerical operations	Foundation of all ML libraries
Scikit-learn	ML model training & eval	Simple API, powerful algorithms, well-documented
Matplotlib	Data visualisation	Standard Python plotting library
Django	Web framework (backend)	Batteries-included, secure, Python-native MVC
HTML/CSS/JS	Frontend UI	Standard web stack, no extra framework needed
SQLite	Default Django database	Zero-config, sufficient for this scale
Joblib/Pickle	Model serialisation	Save trained model to file, load in Django view

3. Machine Learning Pipeline

Step 1: Data Collection

Downloaded the CarDekho dataset from Kaggle. It contains 8,128 rows of real car listings with selling prices in India.

Step 2: Exploratory Data Analysis (EDA)

Used Pandas and Matplotlib to understand the data — checked distributions, found correlations, identified which features most affect price (year and km_driven are strongest predictors).

Step 3: Data Preprocessing

Handled missing values, removed outliers (cars with unrealistic prices), encoded categorical variables (fuel type, seller type, transmission, owner) using Label Encoding / One-Hot Encoding, and split data into 80% training and 20% test sets.

Step 4: Feature Engineering

Created a 'car_age' feature from year. Dropped the original 'year' column after creating this. Normalized km_driven for better model performance.

Step 5: Model Selection & Training

Trained multiple regression models: Linear Regression, Decision Tree, Random Forest, and Gradient Boosting. Evaluated each using R^2 Score and RMSE. Random Forest gave the best results (~94% R^2).

Step 6: Model Evaluation

Final model metrics — R^2 Score: ~0.94, RMSE: ~1.2 Lakhs. This means the model explains 94% of variance in car prices and is off by about ■1.2 Lakhs on average.

Step 7: Model Saving

Used joblib.dump() to serialize the trained model to a .pkl file. This file is loaded once in Django at startup and reused for every prediction request.

Step 8: Django Integration

The saved model is loaded in views.py. When a user submits the form, the input is preprocessed identically to training data, passed to model.predict(), and the result is rendered in result.html.

4. System Architecture & Django Flow

How Django MVC works in this project

Component	File	Responsibility
Model (M)	models.py	Defines database tables (if any). In this project, minimal DB usage — predictions are s
View (V)	views.py	Core logic — receives form data, preprocesses it, calls ML model, returns result to tem
Template (T)	templates/*.html	HTML pages rendered by Django. Receives context variables from views (car name, p
URL Router	urls.py	Maps URLs to view functions. /predict/ → predict() view.
Static Files	static/	CSS, JS, car images. Loaded in templates using {% load static %}.
ML Model	model.pkl	Serialised Random Forest model. Loaded once in views.py using joblib.load().
Settings	settings.py	Django configuration — database, static files, installed apps, secret key.

Request-to-Response Flow

#	Stage	Detail
1	User fills form	home.html — selects car model, year, km driven, fuel type, etc.
2	Form POST request	Browser sends POST to /predict/ URL
3	urls.py routes it	path('predict/', views.predict) — calls the predict() function
4	views.py processes	Extracts form data, encodes categorical features, builds feature array
5	Model predicts	model.predict([features]) returns price in Lakhs
6	Template rendered	result.html receives car_name, price, car_image, and displays them
7	User sees result	Formatted price card with car image shown to user

5. Guide Q&A; — Every Question You Will Face

COLOUR CODE — GREEN: basic/warm-up | BLUE: expected core questions | RED: deep-dive technical | AMBER: trap/trick questions your guide might ask

A. Project Fundamentals

BASIC

Q: What is your project about?

A: It is a machine learning-powered web application that predicts the resale price of used cars in India. The user provides car details like model, year, kilometres driven, fuel type, seller type, transmission, and owner type. A trained Random Forest regression model processes these inputs and returns an estimated selling price in Lakhs.

BASIC

Q: Why did you choose this project?

A: The Indian used car market lacks transparent pricing standards. Buyers and sellers often make uninformed decisions. This project applies machine learning to solve a real, everyday problem with available public data, making it practically relevant and technically substantial for a final year project.

BASIC

Q: What is the dataset you used?

A: I used the CarDekho dataset available on Kaggle. It contains 8,128 records of used car listings with features including car name, year, selling price, km driven, fuel type, seller type, transmission, and owner type. The target variable is the selling price in Lakhs.

EXPECTED

Q: What is the scope and limitation of your project?

A: Scope: Predicts prices for Indian used car models in the dataset range. Works for petrol, diesel, CNG, and electric vehicles. Handles multiple seller types and owner categories. Limitation: Cannot predict prices for car models not in the training data. Does not account for car condition, accident history, or regional price variation. Predictions are estimates, not guarantees.

B. Machine Learning Questions

EXPECTED

Q: What algorithm did you use and why?

A: I used Random Forest Regression. It is an ensemble method that builds multiple decision trees and averages their predictions. I chose it because: (1) it handles non-linear relationships well, (2) it is robust to outliers, (3) it requires minimal feature scaling, and (4) it provided the best R^2 score (~0.94) compared to Linear Regression, Decision Tree, and Gradient Boosting in my experiments.

EXPECTED

Q: What is R^2 score and what did you get?

A: R^2 (R-squared) or the coefficient of determination measures how well the model explains the variance in the target variable. A score of 1.0 means perfect prediction; 0 means the model is no better than predicting the mean. My model achieved approximately 0.94, meaning it explains 94% of the variance in used car prices.

EXPECTED

Q: What is RMSE and what was yours?

A: RMSE — Root Mean Squared Error — is the square root of the average squared differences between predicted and actual values. It is in the same unit as the target variable (Lakhs). My model's RMSE was approximately 1.2 Lakhs, meaning on average the prediction is off by ■1.2 Lakhs.

BASIC

Q: What is the difference between regression and classification?

A: Regression predicts a continuous numerical value — like a price. Classification predicts a discrete category — like 'cheap', 'moderate', 'expensive'. This project is a regression problem because we are predicting an exact price value, not a category.

EXPECTED

Q: How did you handle categorical variables?

A: Categorical variables like fuel type (Petrol/Diesel/CNG), seller type (Individual/Dealer), transmission (Manual/Automatic), and owner type were encoded numerically. I used Label Encoding for ordinal features and One-Hot Encoding for nominal features. The same encoding applied during training is applied during prediction in Django.

EXPECTED

Q: What is overfitting? Did your model overfit?

A: Overfitting occurs when a model learns the training data too well — including noise — and performs poorly on unseen data. I checked for it by comparing training accuracy (~96%) vs test accuracy (~94%). The small gap indicates the model generalizes well and is not significantly overfitting.

EXPECTED

Q: What is train-test split and what ratio did you use?

A: Train-test split divides the dataset into two parts: training data used to fit the model and test data used to evaluate performance on unseen samples. I used an 80-20 split — 80% for training (~6,500 rows) and 20% for testing (~1,600 rows). I used random_state=42 for reproducibility.

DEEP DIVE

Q: Why not use deep learning for this?

A: Deep learning requires very large datasets and significant computational resources to outperform traditional ML on tabular data. With ~8,000 rows and structured tabular features, Random Forest performs comparably or better than deep learning models, trains faster, and is far more interpretable — which is important when explaining predictions.

DEEP DIVE

Q: What is feature importance in Random Forest?

A: Random Forest can calculate how much each feature contributed to the model's predictions by measuring how much each feature reduces impurity across all trees. In my model, 'year' (car age) and 'km_driven' had the highest importance, meaning newer cars with fewer kilometres are the strongest predictors of higher price.

DEEP DIVE

Q: Could you have used cross-validation? Why or why not?

A: Yes, and ideally I should have. K-fold cross-validation (e.g., 5-fold) provides a more robust evaluation by training and testing on multiple different data subsets. It reduces the variance of the performance estimate compared to a single 80-20 split. For this project, a single split was sufficient to demonstrate the concept, but cross-validation would be the professional standard.

C. Django & Web Application Questions

EXPECTED

Q: What is Django and why did you use it?

A: Django is a high-level Python web framework that follows the MVT (Model-View-Template) architecture. I chose it because it integrates naturally with Python ML code, has built-in security features (CSRF protection, SQL injection prevention), handles routing and templating cleanly, and is industry-standard for Python web development.

EXPECTED

Q: Explain the MVT pattern in your project.

A: MVT stands for Model-View-Template. Model handles data structure and database interaction. View contains the business logic — in my project, it receives form data, preprocesses it, calls the ML model, and passes results to the template. Template is the HTML file that renders the final page the user sees. URLs map incoming requests to the correct view function.

EXPECTED

Q: How does the ML model get used inside Django?

A: The trained model is serialized using joblib.dump() and saved as a .pkl file during the training phase. In views.py, I load it once at module level using model = joblib.load('model.pkl'). When a user submits the prediction form, the view extracts inputs, builds a feature array identical to training data format, calls model.predict([features]), and passes the result to the template.

DEEP DIVE

Q: What is CSRF and how does Django handle it?

A: CSRF — Cross-Site Request Forgery — is an attack where a malicious website tricks a user into submitting a form to another site they are logged into. Django handles this by generating a unique CSRF token for each session and embedding it in forms via {% csrf_token %}. The server validates this token on every POST request, rejecting any that do not match.

EXPECTED

Q: What happens when a user enters a car not in your dataset?

A: I have a KNOWN_CARS list in views.py containing all car models the model was trained on. If the submitted car name is not in this list, the view renders a 'not_available.html' page explaining that the model cannot predict for that car, and suggests similar available models.

D. Trap & Tricky Questions

These are questions designed to catch students who built the project without understanding it. Read these answers carefully — they are your most important preparation.

TRAP Q

Q: Can you show me the code for your model training?

A: Yes. The training is done in a Jupyter notebook. Key lines: pd.read_csv() to load data, LabelEncoder() or get_dummies() for encoding, train_test_split(X, y, test_size=0.2, random_state=42) for splitting, RandomForestRegressor(n_estimators=100, random_state=42) to define the model, model.fit(X_train, y_train) to train, r2_score() and mean_squared_error() to evaluate, joblib.dump(model, 'model.pkl') to save. I can walk through each step.

TRAP Q

Q: What is n_estimators in Random Forest?

A: n_estimators is the number of decision trees in the forest. I used 100 trees. More trees generally improve accuracy and reduce variance but increase training time. After about 100-200 trees, the improvement becomes marginal. I tested 50, 100, and 200 — 100 gave the best trade-off.

TRAP Q

Q: Your model is 94% accurate — what is the remaining 6% error from?

A: The remaining error comes from: (1) features not captured in the dataset such as car condition, accident history, geographic location, and service records; (2) natural price variation between sellers even for identical cars; (3) model limitations — no model achieves 100% on real-world data. This is expected and acceptable for a regression task of this complexity.

TRAP Q

Q: If I give 'km_driven = 0', what will your model predict?

A: A very new or unsold car with 0 km driven and a recent year would predict a price close to the ex-showroom price. The model has seen very low km_driven values in training data for nearly-new cars. It is a valid input. However, the model was trained on used car data, so predictions for brand-new cars (0 km) are extrapolations and may not be as accurate.

TRAP Q

Q: Why didn't you normalise / scale your features?

A: Random Forest does not require feature scaling because it is a tree-based model. Decision trees split on feature thresholds, not on distances or magnitudes, so having features on different scales does not affect the splits. Scaling is important for distance-based models like KNN or gradient-based models like Linear Regression with regularization.

TRAP Q

Q: How would you improve this project with more time?

A: Four clear improvements: (1) Add city/location as a feature — prices vary 15-20% between metros. (2) Use cross-validation for more robust evaluation. (3) Add more cars to the training data by scraping current listings. (4) Deploy to a cloud server (Heroku, Railway) so it is publicly accessible. (5) Add an EMI calculator and depreciation chart for better user utility.

6. Key Terms — Quick Reference Card

Term	Definition
Machine Learning	Teaching computers to make predictions from data without explicit rules.
Regression	Predicting a continuous number (like a price), not a category.
Random Forest	Ensemble of decision trees. Averages predictions to reduce error.
Decision Tree	Flowchart-like model that splits data based on feature thresholds.
R² Score	How well the model explains variance in data. 1.0 = perfect.
RMSE	Average prediction error in the same unit as target (Lakhs here).
Overfitting	Model memorises training data and fails on new data.
Feature Engineering	Creating new features from existing ones (e.g., age from year).
Label Encoding	Converting text categories to numbers (Petrol=0, Diesel=1, CNG=2).
One-Hot Encoding	Creating binary columns for each category value.
Train-Test Split	Dividing data — training set to learn, test set to evaluate.
Django MVT	Model-View-Template: Django's architecture pattern.
CSRF Token	Security token in forms to prevent cross-site request forgery.
Pickle / Joblib	Python tools to save and load ML models to/from files.
n_estimators	Number of trees in a Random Forest. I used 100.
Feature Importance	How much each input variable contributed to the model's predictions.
Hyperparameter	Model settings set before training (n_estimators, max_depth, etc).
Cross-Validation	More robust evaluation by testing on multiple data subsets (K-fold).